

Modern Cloud Computing

Lecture 6: Delivery Models I: IaaS and PaaS

Chongzhou Fang

Rochester Institute of Technology

Today's Topics

- Service Model Comparison
- IaaS
- PaaS
- IaaS vs PaaS

Service Model Comparison

Service Models

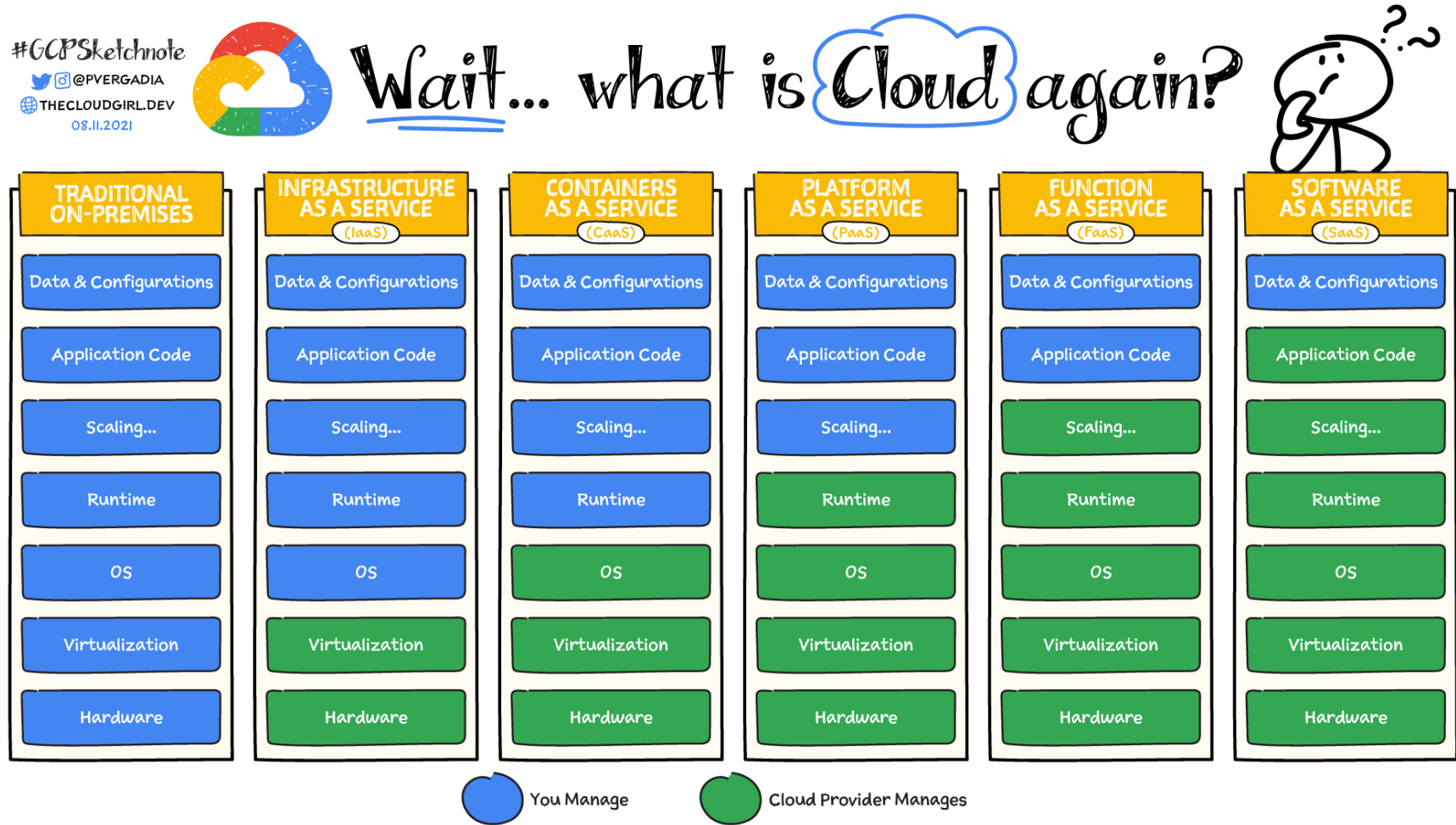
Cloud computing:

- Physical infrastructure
- Virtualization and resource pooling
- Operating systems and runtimes
- Middleware and databases
- Application code and data

Different service models define different control boundaries.

Sources: IBM, Google Cloud, Microsoft Learn, NIST.

Service Model Comparison



laaS

IaaS: Definition

Infrastructure as a Service (IaaS) provides on-demand access to cloud-hosted compute, storage, and networking resources.

In IaaS, the provider manages the foundational infrastructure, while users manage what runs above it.

Typical user responsibilities:

- Operating systems
- Runtime environments
- Middleware
- Applications
- Data and application-level security

Sources: IBM, Google Cloud, NIST.

IaaS as a Virtual Data Center

IaaS is often easiest to understand as a programmable virtual data center.

Users can request:

- Virtual machines or bare-metal servers
- File storage
- Virtual Network

IaaS Components

Core resources

- Compute
- Storage
- Networking

Control-plane services

- Scheduling
- Placement
- Provisioning
- Metering
- Monitoring

IaaS Compute

IaaS configurations may include:

- Virtual machines vs. Dedicated bare-metal servers
- GPUs and accelerators
- vCPU and memory
- Availability zones and regions

Users choose instance sizes and locations based on workload needs.

Tradeoff: more customization gives more responsibility for configuration and maintenance.

Virtualization in IaaS

Virtualization enables multiple virtual machines to share one physical server.

A hypervisor:

- Creates isolated virtual hardware environments
- Allocates CPU, memory, storage, and network resources
- Helps enforce isolation between tenants
- Enables VM lifecycle operations such as start, stop, snapshot, and migration

IaaS depends heavily on virtualization and resource multiplexing.

VM Lifecycle

A simplified VM lifecycle:

- Choose image
 - Configure network/security
 - Schedule to host
 - Boot VM
 - Attach volumes
 - Monitor and scale
 - Snapshot / stop / terminate

IaaS Networking

IaaS networking is typically software-defined.

Users can create and manage:

- Virtual networks and subnets
- IP addressing
- Routers and gateways
- Security groups and firewall rules
- VPNs and private connectivity
- Load balancers

IaaS Storage

IaaS commonly offers several storage models.

Storage type	What it provides	Common use
Block storage	Volumes attached to VMs	Databases, file systems
Object storage	Objects in buckets/containers	Backups, media, data lakes
File storage	Shared file system access	Shared content, legacy apps

Storage choice affects performance, consistency, cost, and application design.

IaaS Identity and Tenancy

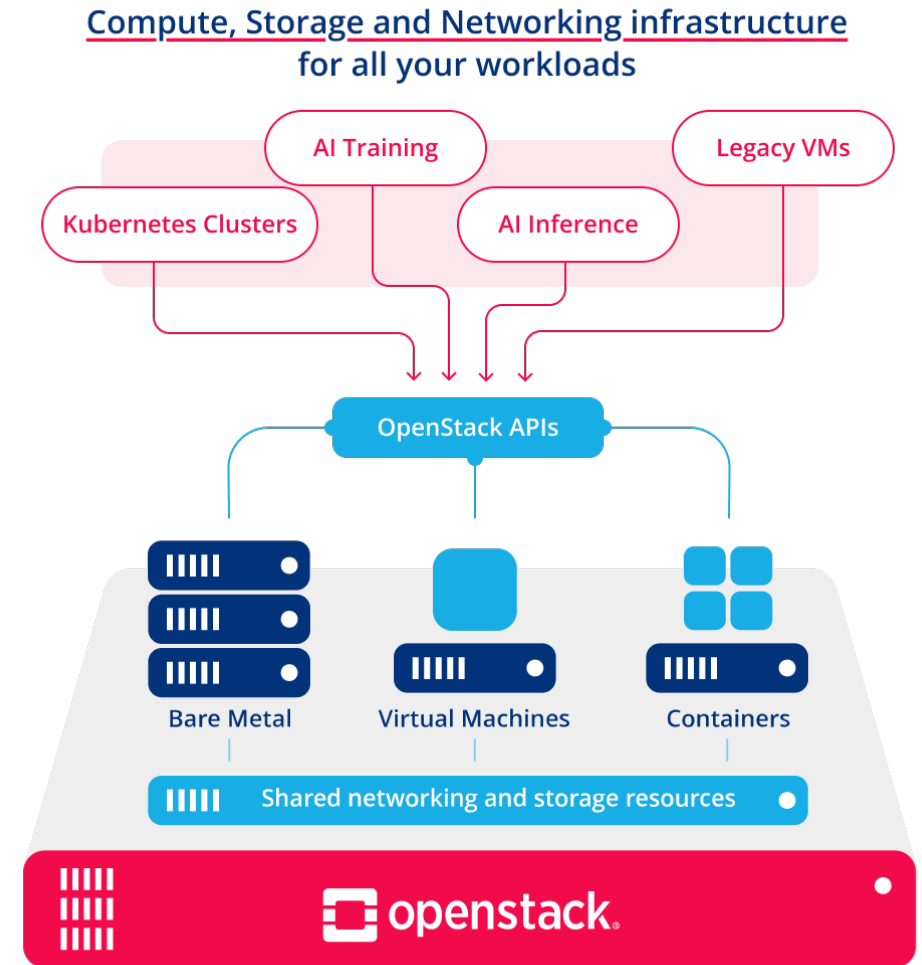
Identity services support:

- Authentication
- Authorization
- Projects/tenants
- Roles and policies
- Service accounts
- API credentials

Without identity and access control, IaaS cannot safely expose infrastructure APIs to many users.

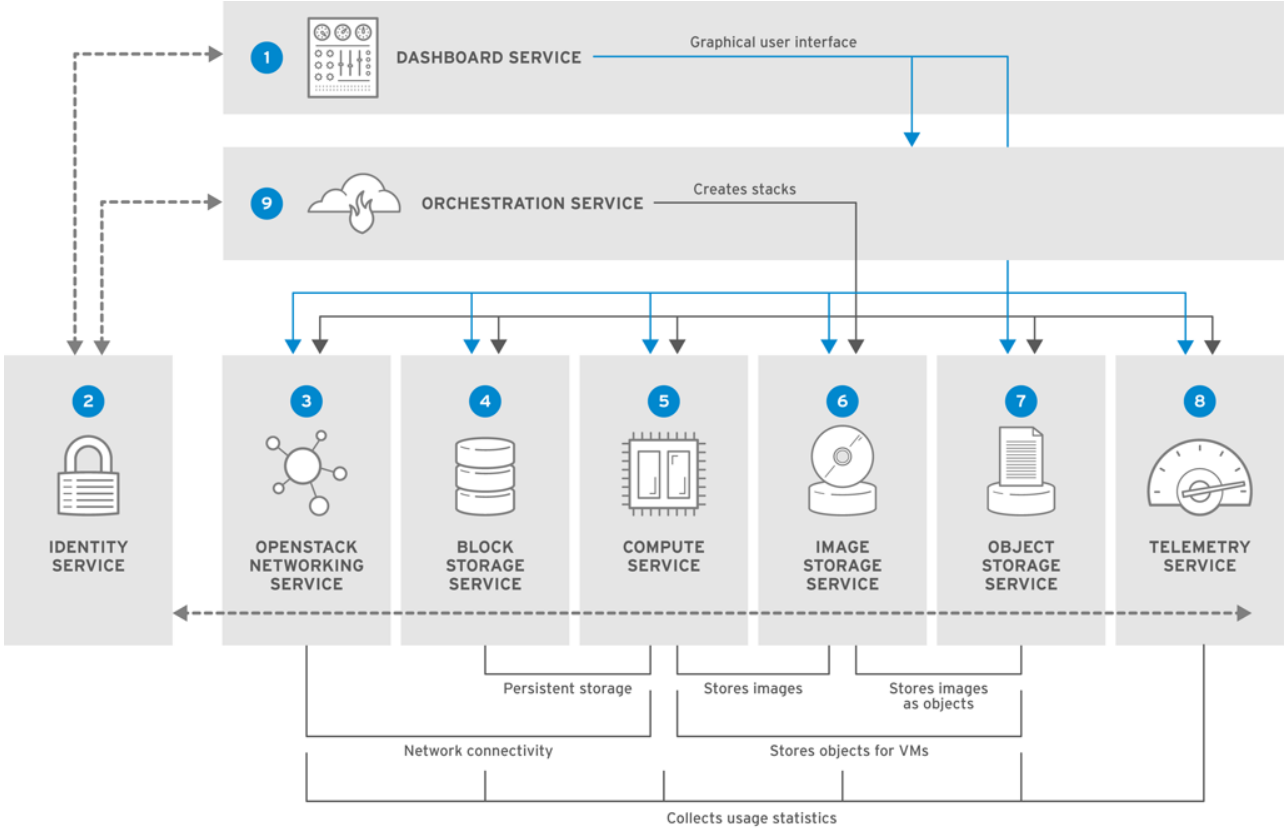
OpenStack

OpenStack is a cloud operating system that controls large pools of compute, storage, and networking resources throughout a datacenter



OpenStack Component Map

Function	OpenStack service
Compute	Nova
Networking	Neutron
Block storage	Cinder
Object storage	Swift
Images	Glance
Identity	Keystone
Dashboard	Horizon
Orchestration	Heat
Telemetry	Ceilometer



RHELOSP_347192_1015

OpenStack Compute: Nova

Nova is responsible for compute provisioning and lifecycle management.

Conceptually, Nova handles:

- Requests to create, delete, stop, and start instances
- Scheduling instances onto compute hosts
- Talking to the hypervisor layer
- Coordinating with image, network, identity, and placement services

Nova is the IaaS compute control plane.

OpenStack Networking: Neutron

Neutron provides network connectivity as a service.

It helps users create cloud networks with:

- Virtual routers and subnets
- Floating IPs
- Security groups
- Network backends and plugins

This allows the cloud to support different network designs without hard-wiring one physical topology into every tenant.

OpenStack Storage: Cinder and Swift

Cinder

Block storage service

- Creates volumes
- Attaches volumes to VMs
- Supports snapshots
- Useful for databases and persistent disks

Swift

Object storage service

- Stores objects
- Uses containers/buckets
- Useful for backups, media, and unstructured data
- Designed for scale-out storage

Other OpenStack Services

- **Glance:** stores and manages VM images
- **Keystone:** provides identity services
- **Horizon:** web dashboard for users and operators
- **Heat:** orchestration using infrastructure templates
- **Ceilometer:** telemetry and metering
- **Ironic:** bare-metal provisioning

IaaS Use Cases

IaaS is useful when users need infrastructure flexibility.

Common examples:

- Hosting custom applications
- Development and testing environments
- Backup and disaster recovery
- Big data analytics
- High-performance computing
- Hybrid cloud migration
- Running legacy workloads with specific OS/runtime needs

IaaS Benefits

IaaS is attractive because it offers:

- High control over infrastructure configuration
- Fast provisioning compared with purchasing hardware
- Elastic scaling for variable demand
- Usage-based pricing
- Access to specialized hardware and global regions
- Support for hybrid and multicloud strategies

IaaS is closest to owning infrastructure without owning the data center.

IaaS Limitations

IaaS does not eliminate operations work.

Users may still need to manage:

- OS patching and hardening
- Runtime and middleware updates
- Application deployment
- Backups and disaster recovery design
- Network configuration
- Security groups and access controls
- Monitoring and incident response

IaaS shifts hardware responsibility to the provider, but much of the system stack remains yours.

PaaS

PaaS: Definition

Platform as a Service (PaaS) provides an environment to develop, deploy, and operate applications.

A PaaS provider typically manages:

- Hardware and virtualization
- Operating systems
- Runtime environments
- Middleware
- Application platform services
- Deployment, monitoring, scaling, and management capabilities

The user focuses mainly on **application code and data**.

Sources: CSCC Practical Guide to PaaS, IBM, Google Cloud.

PaaS as an Application Platform

PaaS gives developers a managed application environment.

It commonly includes:

- Runtimes for programming languages
- Build and packaging systems
- Web/application servers
- Databases and service integrations
- Routing and load balancing
- Autoscaling
- Logging, metrics, and debugging tools
- Developer-facing CLI, GUI, or API

It provides:

- Development and deployment tools
- Application lifecycle management
- Runtime and middleware services
- Monitoring and management
- Security and identity integration
- Cloud-native scaling and resilience support

PaaS Developer Experience

A typical PaaS workflow:

```
cf login
cf push my-app
cf create-service postgres standard my-db
cf bind-service my-app my-db
cf scale my-app -i 3
cf logs my-app
```

The developer does not manually provision VMs, install OS packages, configure load balancers, or wire service credentials by hand.

PaaS Buildpacks

A tool that detects source code and builds a runnable app image.

- Detect whether the app is compatible
- Supply dependencies
- Finalize the app for launch
- Return metadata describing how the app should run

PaaS Service Binding

Applications often need backing services:

- Databases
- Message queues
- Caches
- Object stores
- Monitoring services
- Authentication providers

PaaS platforms commonly expose these through a service catalog or marketplace.

A service binding gives the app credentials and connection information.

PaaS Routing

A PaaS must route external traffic to the correct app instance.

Routing responsibilities include:

- Mapping domains/routes to apps
- Load balancing across app instances
- Updating routes when instances move or restart
- Supporting health checks
- Integrating with external load balancers

PaaS Scaling and Health Management

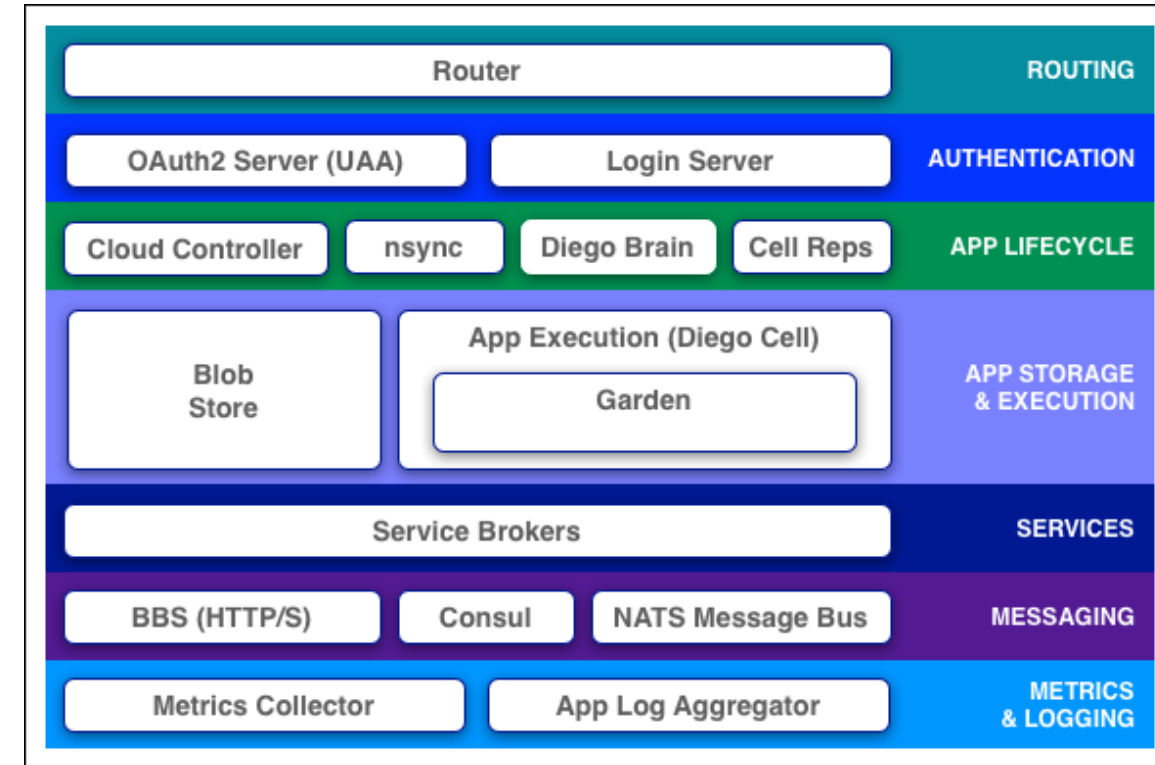
PaaS platforms support elasticity at the application level.

They may provide:

- Manual horizontal scaling
- Autoscaling rules
- Health checks
- Instance restart after failure
- Rolling or blue-green deployments
- Placement across hosts for resilience

Cloud Foundry Runtime Architecture

Runtime subsystem	Main components	Role
Routing	Router / Gorouter	Routes external traffic to Cloud Controller and hosted applications
Authentication	OAuth2 Server / UAA, Login Server	Provides identity management, login, authentication, and authorization
App lifecycle	Cloud Controller, nsync, Diego Brain, Cell Reps	Coordinates app deployment, desired state, scheduling, and app-instance monitoring
App storage and execution	Blobstore, Diego Cell, Garden	Stores app packages/buildpacks and runs app instances in Garden containers
Services	Service Brokers	Exposes backing services such as databases, caches, queues, and third-party services
Messaging	BBS, Consul, NATS Message Bus	Stores Diego state, supports service discovery/coordination, and distributes platform messages
Metrics and logging	Metrics Collector, App Log Aggregator / Loggregator	Collects platform metrics and streams application logs/events



PaaS Benefits

PaaS can help organizations:

- Accelerate development and deployment
- Reduce infrastructure management work
- Standardize runtime environments
- Improve developer productivity
- Provide built-in scaling and monitoring
- Encourage cloud-native application design
- Reduce time from code to production

PaaS Limitations

PaaS also introduces tradeoffs:

- Less control over runtime and infrastructure
- Possible restrictions on languages or frameworks
- Platform-specific deployment patterns
- Vendor/platform lock-in
- Debugging complexity when the platform abstracts details

IaaS vs PaaS

IaaS vs. PaaS: Technical Comparison

Dimension	IaaS	PaaS
Main abstraction	Infrastructure resources	Application platform
User controls	OS, runtime, middleware, app, data	App, data, platform config
Provider controls	Physical infra + virtualization	Infra + OS + runtime + middleware
Deployment unit	VM, volume, network	App, container, service binding
Scaling target	Instances/resources	App instances/services
Best for	Custom stacks, legacy apps, full control	Fast app delivery, standard runtimes
Main risk	Operational burden	Reduced control / lock-in

Responsibility Boundary

IaaS users manage

- OS patches
- Runtime versions
- Middleware config
- Network rules
- VM hardening
- Application deployment
- Backup strategy

PaaS user worries about

- Application code
- Data model
- Service usage
- App-level secrets
- Scaling policy
- Runtime compatibility
- Platform constraints

Choosing IaaS

Choose IaaS when:

- You need custom OS, kernel, or runtime control
- The application is legacy or hard to replatform
- You need specialized networking or security appliances
- You need full control over middleware
- Performance tuning requires low-level choices

Choosing PaaS

Choose PaaS when:

- Developer speed is more important than low-level control
- The application fits supported runtimes or containers
- The organization wants standardized deployment workflows
- The app is cloud-native or being refactored
- Platform operators can manage the shared application platform

Example: Ecommerce Application

IaaS approach

- Provision VMs
- Install OS packages
- Configure web servers
- Set up database
- Configure load balancer
- Manage patches and backups

PaaS approach

- Push app code
- Bind database service
- Configure routes
- Scale app instances
- Stream logs
- Use platform health checks

Summary

- IaaS provides virtualized infrastructure through APIs.
- PaaS provides an application platform that hides much of the infrastructure.
- IaaS gives more control but leaves more operational work to the user.
- PaaS improves developer productivity but imposes platform assumptions.
- OpenStack is a concrete example of IaaS architecture.
- Cloud Foundry is a concrete example of PaaS architecture.

References

- IBM: "IaaS, PaaS and SaaS: What's the difference?"
- Google Cloud: "PaaS vs IaaS vs SaaS: What's the difference?"
- Microsoft Learn: "Describe cloud service types"
- NIST SP 500-292: "NIST Cloud Computing Reference Architecture"
- Red Hat OpenStack Platform 8: "Architecture Guide"
- Cloud Standards Customer Council: "Practical Guide to Platform-as-a-Service"
- Cloud Foundry Docs: overview, Diego architecture, buildpacks, routing, services, logging